1. Vad är en databas?

Vi människor har alltid behövt hålla reda på saker. Vilka varor finns i lager? Vem äger ett hus? Hur mycket pengar finns på ett konto? Vilka elever går i en viss klass?

Vi kan skriva ner informationen på papper, spara den i filer eller hålla den i minnet. Men när mängden information växer behöver vi kunna organisera den så att den blir enkel att hitta, använda och förändra.

Det är i grunden vad databaser handlar om.

Grund

Data, nycklar och värden

Vi kan börja med något mycket enkelt:

```
Förnamn: Benjamin
```

Här har vi en **nyckel** (`Förnamn`) och ett **värde** (`Benjamin`).

Nyckeln beskriver vad informationen handlar om och värdet innehåller själva uppgiften.

Om vi samlar flera sådana uppgifter kan vi beskriva exempelvis en kund i en bokhandel:

```
Kund:
    Förnamn: Benjamin
    Efternamn: Berglund
    Telefon: 0123–456789
    Adress:
        Gata: Hemmagatan 1
        Postnummer: 124 56
        Ort: Hemmavan
```

Nu har vi skapat en enkel datastruktur. Vi har också grupperat några av uppgifterna: gatan, postnumret och orten beskriver tillsammans en adress.

Samma information kan representeras på flera olika sätt.

I JavaScript kan vi exempelvis skriva:

```
const customer = {
    firstName: "Benjamin",
    lastName: "Berglund",
    phone: "0123-456789",
    address: {
        street: "Hemmagatan 1",
        postalCode: "124 56",
        city: "Hemmavan"
    }
};
```

Men vi skulle också kunna organisera informationen i en tabell.

Tabeller

Förnamn	Efternamn	Telefon	Ort
Benjamin	Berglund	0123-456789	Hemmavan
Olof	Olofsson	0231-564789	Storfors
Anaïs	Martin	+43 21-976 542	Loire

Här beskriver varje **rad** en kund, och varje **kolumn** en viss typ av uppgift om kunden.

Vi kan förstå tabellen genom tre enkla begrepp:

- **Tabell:** En samling uppgifter som hör ihop.
- **Rad:** En enskild post, exempelvis en kund.
- **Kolumn:** En viss egenskap hos posterna, exempelvis förnamn.

I en relationell databas bygger vi upp vår information med sådana tabeller.

Vad gör detta till en databas?

En tabell i ett dokument eller kalkylblad är inte automatiskt en databas.

En **databas** är en organiserad samling data som kan lagras, hanteras och användas. För att arbeta med den använder vi vanligtvis ett **databashanteringssystem** (DBMS, Database Management System).

I relationella databaser använder vi frågespråket **SQL (Structured Query Language)** för att läsa, lägga till, förändra och ta bort information. SQL är utformat för att likna vanlig engelska, samtidigt som språket har en tydlig struktur som datorer kan tolka.

Exempel på databashanteringssystem som använder SQL är MySQL, PostgreSQL, SQLite och Microsoft SQL Server.

Databashanteraren hjälper oss bland annat att:

- Lagra och hämta information.
- Lägga till, ändra och ta bort uppgifter.
- Söka efter information som uppfyller vissa villkor.
- Kontrollera att informationen följer bestämda regler.
- Hantera att flera användare arbetar med samma information.

Skillnaden mellan databasen och databashanteraren kan jämföras med skillnaden mellan ett dokument och programmet som hanterar dokumentet.

Att ställa frågor till informationen

Anta att bokhandeln har en databas med tusentals kunder.

Vi kanske vill veta vilka kunder som bor i Hemmavan, eller vilka som har ett efternamn som börjar med B.

Vi skulle kunna läsa igenom alla poster manuellt, men det vore knappast praktiskt.

I stället kan vi ställa en fråga till databasen.

Med SQL kan en sådan fråga se ut så här:

```
SELECT *
FROM customers
WHERE city = 'Hemmavan';
```

Vi behöver inte förstå syntaxen än. Det viktiga är att vi beskriver **vilken information vi vill ha**, snarare än hur databashanteraren ska leta efter den.

Detta är en grundläggande idé bakom SQL.

Relationer mellan data

Låt oss anta att bokhandeln också behöver hålla reda på kundernas adresser.

Vi skulle kunna lagra gatuadress, postnummer och ort tillsammans med varje kund. Men vad händer när en kund vill ha en separat leveransadress? Eller när adressuppgifter behöver förändras?

Ett annat sätt är att lagra kunder och adresser i separata tabeller och beskriva hur de hör ihop.

Kunder (customers)

id	name
1	Benjamin
2	Olof

Adresser (addresses)

id	customer_id	street	city
1	1	Hemmagatan 1	Hemmavan
2	1	Storgatan 4	Storfors
3	2	Parkvägen 2	Storfors

Kolumnen `customer_id` innehåller kundens id och visar vilken kund varje adress hör till.

Nu kan en kund ha flera adresser, utan att vi behöver skapa fler adresskolumner i kundtabellen.

Detta är en av de viktigaste principerna bakom relationella databaser: vi kan organisera uppgifter om olika saker var för sig, samtidigt som vi bevarar sambanden mellan dem.

Vi kommer att återkomma till hur relationer fungerar i senare kapitel.

Avancerat

En databas är en modell av verkligheten

När vi lagrar information beskriver vi inte hela verkligheten. Vi väljer ut de egenskaper och samband som är relevanta för vårt syfte.

En kund har betydligt fler egenskaper än namn och adress. Men om vi bygger ett kundregister kanske det är allt vi behöver veta.

När bokhandeln senare ska hantera beställningar, produkter och betalningar behöver vi andra uppgifter och andra samband.

Det innebär att en databas inte bara är ett sätt att lagra information. **Den är också en modell av den del av verkligheten vi vill beskriva.**

Modellen påverkar vad vi senare kan fråga efter, beräkna och dra slutsatser om.

Är alla databaser relationella?

Nej.

Information kan organiseras på olika sätt, och olika databastyper är lämpliga för olika problem.

Exempelvis:

Databastyp	Grundläggande idé
Relationell	Information organiseras i tabeller med definierade samband.
Dokumentorienterad	Information organiseras som dokument, exempelvis JSON.
Nyckel-värde	Information hämtas och lagras med hjälp av identifierande nycklar.
Grafdatabas	Information representeras som noder och relationer mellan dessa.

Det finns många fler varianter och kombinationer.

Den här boken handlar framför allt om **relationella databaser och SQL**, men många av resonemangen kring hur vi representerar och organiserar information är relevanta även för andra databastyper.

Data och datatyper

Hittills har vi beskrivit information med text och tabeller. Men alla värden är inte av samma slag.

Exempelvis:

```
Namn: Anna
Ålder: 27
Pris: 149.50
Registrerad: 2026-01-15
Aktiv: Ja
```

Vi vill kunna göra olika saker med dessa värden.

Ett namn kan vi jämföra eller söka efter. En ålder kan vi räkna med. Ett datum kan vi jämföra med andra datum.

Databaser använder därför **datatyper** för att beskriva vilken sorts värden som kan lagras i en kolumn.

Det hjälper databashanteraren att tolka och bearbeta informationen på ett förutsägbart sätt.

Vi kommer att gå närmare in på datatyper när vi börjar skapa egna tabeller.

Databasens ansvar

Vi har redan sett att en databas kan lagra information och besvara frågor.

Men vad händer om två användare försöker förändra samma uppgift samtidigt? Eller om en ändring avbryts innan den är färdig?

En databashanterare behöver kunna hantera sådana situationer utan att informationen blir motsägelsefull eller går förlorad.

Det är bland annat därför vi använder databashanterare i stället för att bara skriva och läsa vanliga textfiler.

Senare i boken kommer vi att undersöka hur databaser upprätthåller dataintegritet, hanterar transaktioner och gör information tillgänglig för flera användare.

Reflektion

1. Vad är egentligen skillnaden mellan att lagra information och att organisera information?
2. Hur påverkas en datamodell av vad vi vill använda informationen till?

3. Finns det situationer där en vanlig fil är en bättre lösning än en databas?